

Feature	RDBMS	NoSQL	NewSQL
Schema-less	-	yes	-
High performance	-	yes	yes
Horizontally scalable	-	yes	yes
Relational model	yes	-	yes
SQL	yes	-	yes
ACID transactions	yes	-	yes

Figure 6: NewSQL compared with RDBMS and NoSQL

```
create table student( rollno BIGINT, name VARCHAR(255),
school CHAR(64))
```

Now, if you create an entry:

```
insert into student values( 'khushi', '2628', 'manu high')
```

The query will generate an error since the type(s) of the data does not match the schema.

Since the student type is a record, a schema-less database would support non-uniform types, i.e., each of the following can be stored:

```
{'name' : 'khushi'}
{'name' : 'khushi', 'rollno' : '2628'}
{'name' : 'khushi', 'rollno' : '2628', 'school' : 'manu high'}
```

This is the flexibility of a NoSQL data store; however, there are three points to note here.

1. The information about the type and structure of the data shifts to the code that you write.
2. The implicit schema (a data type) is a data structure in your code. Depending on the language, it would be represented as a struct, class, a hash, a dictionary, an associative array or a map that is also the representation of the schema-less data structure in memory.
3. As a programmer, you are going to encounter errors for any data that does not match the data type.

A useful approach is to model a set of accessory methods and related logic, which provide a facade to the schema-less data structure. This one level of indirection decouples other parts of your code from the core structure.

How an RDBMS goes schema-less

Building on the discussion above, we can see that a non-uniform type is characterised by an indeterminate number of key-value pairs.

As a result:

1. A key may refer to a JSON object or an array of multiple key-value pairs.
2. A key may have additional custom data.

All the above can be modelled with additional tables in an RDBMS using custom columns in a table, a TEXT field to embed semi-structured data like JSON, and an attribute table that joins the custom data.

This is a pragmatic approach to support ad hoc schema-less data over a relational schema.

A schema as a contract

So from the point of view of a service, what exactly is a schema?

XML vocabulary has DTD, JSON layout maps for the schema, and an RDBMS has its schema defined in terms of integrity constraints. In more generic terms, the notion of database schema plays the same role as the notion of theory in predicate calculus.

A schema, in some sense, is a documented form of the contract that a particular component implements. In other words, as an architect would say, a schema is an *interface*!

NoSQL PostgreSQL

Let us look at an example of extensible schema in the context of PostgreSQL, an ORDBMS (Object Relational Database Management System). Traditionally, PostgreSQL has been well known for rich SQL and transactional interfaces and capabilities.

With the release of version 9.4, PostgreSQL has stepped into the world of Not only SQL by bringing in two new features—HStore and JSONB.

JSONB is a binary version of the JSON format, which supports indexing and a rich set of JSON operators that include *containment* and *existence*. JSONB supports *btree* and hash indexes. GIN or generalised inverted indexes can be used to efficiently search for keys or key-value pairs occurring within a large number of JSONB documents.

Let us look at the example of the Docker inspection JSON fragment we discussed in the section on semi-structured data, and see how we can use the JSON operators of PostgreSQL.

Let's assume that *dockerimages* refers to the images and *dockerinst* refers to the JSON fragment for an image.

```
SELECT dockerinst->'Config', dockerinst->'Container'
FROM   dockerimages
WHERE  dockerinst @> '{"Image" : "d9065eab448785cb1377f3
81135f1e345d31883ae39bb646e9129ea9b3b05b3"}
```

We have accessed specific fields or array chunks in a JSON fragment using the familiar SQL interface. Next, we explore the approach PostgreSQL adopts to support key-value pairs within a single PostgreSQL value.

The *hstore* module implements the *hstore* data type for storing sets of key-value pairs. In other words, *hstore* is a key-value store within PostgreSQL.

Since keys and values are simple text strings, this will be useful for working with semi-structured data and rows that have multiple attributes. Here is an example that highlights how to use the feature.

To install the *hstore* module, you need to run this query in the database you wish to use *hstore* on: